CSE 242 Assignment 3

Name: Mihir Mandar Ranadive

CRUZ ID: mranadiv

Question1: Hard Margin Linear SVM

(a)

```
import matplotlib.pyplot as plt

SVM_X = ((1, 1), (1, 2), (2, 1), (0, 0), (1, 0), (0, 1))
SVM_Y = (1, 1, 1, -1, -1, -1)

fig, ax = plt.subplots(figsize=(6, 6))

# Plot positive and negative points
for label, color, marker in ((1, "blue", "o"), (-1, "orange", "s")):
    points = [x for x, y in zip(SVM_X, SVM_Y) if y == label]
    ax.scatter(
        [x[0] for x in points],
        [x[1] for x in points],
        color=color,
        marker=marker,
        s=100,
        label=f"y = {label:+d}"
    )


for point_id, (x1, x2) in enumerate(SVM_X, start=1):
    ax.annotate(
        str(point_id),
        (x1, x2),
        xytext=(7, 7),
        textcoords="offset points"
    )

ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Assignment 3: SVM Training Data")
ax.set_xlim(-0.5, 2.5)
ax.set_ylim(-0.5, 2.5)

ax.set_aspect("equal")
ax.grid(alpha=0.3)
ax.legend()

plt.savefig("svm_data.png", dpi=180)
plt.show()
```




```
def identify_support_vectors():
    return [1, 5, 6]

print("Support Vector Point IDs:", identify_support_vectors())
```

```
Support Vector Point IDs: [1, 5, 6]
```

The support vectors are points 1, 5, and 6. Support vectors are the data points closest to the separating hyperplane. Point 1 belongs to the positive class, while points 5 and 6 belong to the negative class. These points determine the position of the maximum-margin separating boundary.

(b)

```
#Upload Derivation Img
from IPython.display import display, Image

display(Image(filename='CSE 242 Assignment3 Q1b.jpeg'))
```

CSE 242 Assignment 3

Mihir Mandar Ranadive

(Q1 b) Mathematical Derivation:

General equation for linear SVM is : $w_1x_1 + w_2x_2 + b = 0$

For support vectors, the hard margin SVM condition is $y_i(w^T x_i + b) = 1$

From a), support vectors are points 1, 5, 6

Point	Co-ordinates	Class
1	(1, 1)	+1
5	(1, 0)	-1
6	(0, 1)	-1

Substitute each point in the general equation:

$w_1 + w_2 + b = 1$ ___ (1)

$w_1 + b = -1$ ___ (2)

$w_2 + b = -1$ ___ (3)

(3) - (2) $\Rightarrow w_1 = w_2$

Put in (1) and (2) $\Rightarrow 2w_1 + b = 1$ ___ (4)

$w_1 + b = -1$ ___ (5)

(4) - (5) $\Rightarrow w_1 = 2$

$\therefore w_2 = 2 \Rightarrow b = -3$

$\therefore 2x_1 + 2x_2 - 3 = 0$ → Final decision boundary

Now, Margin $= \frac{1}{\|w\|}$

$\|w\| = \sqrt{w_1^2 + w_2^2} = \sqrt{2^2 + 2^2}$

$= \sqrt{8} = 2\sqrt{2}$

$\therefore$ Margin $= \frac{1}{2\sqrt{2}}$

$\therefore$ Margin $= 0.354$

Full margin width $= \frac{2}{\|w\|} = 2 * (0.354)$

Full margin width = 0.707

```
import numpy as np

def derive_svm_separator():

    # SVM weights
    weights = (2.0, 2.0)

    # Intercept
```

```
    bias = -3.0

    # Calculate geometric margin
    norm_w = np.linalg.norm(weights)

    geometric_margin = 1 / norm_w

    return weights, bias, geometric_margin


# Display results
weights, bias, margin = derive_svm_separator()

print("Weights:", weights)
print("Bias:", bias)
print("Decision Boundary: 2x1 + 2x2 - 3 = 0")
print("Geometric Margin:", margin)
print("Full Margin Width:", 2 * margin)
```

```
Weights: (2.0, 2.0)
Bias: -3.0
Decision Boundary: 2x1 + 2x2 - 3 = 0
Geometric Margin: 0.35355339059327373
Full Margin Width: 0.7071067811865475
```

The maximum-margin decision boundary is obtained using the three support vectors. By substituting their coordinates into the hard-margin SVM equation, we get the weights w1 = 2, w2 = 2 and intercept b = -3. Therefore, the decision boundary is 2x1 + 2x2 - 3 = 0.

The geometric margin is calculated using 1/||w||, which gives approximately 0.3536. The full margin width between the supporting boundaries is approximately 0.7071.

Question2: K-means

(a)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from pathlib import Path
```

```
def update_assignments(data, centroids):

    # Calculate distances between data points and centroids
    distances = np.linalg.norm(
        data[:, np.newaxis, :] - centroids[np.newaxis, :, :],
        axis=2
    )

    # Assign each point to its nearest centroid
    assignments = np.argmin(distances, axis=1)

    return assignments
```

```
 def update_centroids(data, centroids, assignments):

    k = centroids.shape[0]

    # Create a copy to avoid modifying original centroids
    new_centroids = centroids.copy()

    for i in range(k):

        # Get points belonging to cluster i
        cluster_points = data[assignments == i]

        # Update centroid if cluster is not empty
        if len(cluster_points) > 0:
            new_centroids[i] = np.mean(cluster_points, axis=0)

        # If empty, retain previous centroid

    return new_centroids
```

The function calculates the mean of all points assigned to each cluster and updates its centroid. If a cluster has no assigned points, its previous centroid is retained to avoid errors.

(b)

```
def make_toy_data():

    return make_blobs(
        n_samples=4000,
        centers=4,
        cluster_std=0.60,
        random_state=0
    )


data, true_labels = make_toy_data()

print("Dataset shape:", data.shape)
print("Number of data points:", len(data))
```

```
Dataset shape: (4000, 2)
Number of data points: 4000
```

```
def get_initial_clusters(k, data, rng):

    return data[
        rng.integers(0, data.shape[0], size=k)
    ].copy()
```

```
def kmeans(data, centroids, max_iterations=100):

    for _ in range(max_iterations):

        assignments = update_assignments(data, centroids)

        centroids = update_centroids(
            data, centroids, assignments
        )

    # Final assignment
    assignments = update_assignments(data, centroids)

    return centroids, assignments
```

As the value of K increases, the number of clusters increases and the data gets divided into smaller groups. When K=2 or K=3, some of the four original groups are combined. When K=4, the clustering can represent the four original groups more accurately. When K=6 or K=10, some original groups are divided into smaller clusters. Therefore, selecting an appropriate value of K is important for obtaining meaningful clustering results.

Question 3: Noisy-label SVM and K-NN repair

(a)

```
import numpy as np
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score


# Generate clean dataset
def generate_clean_split(rng, samples_per_class=1000):

    positive = rng.multivariate_normal(
        (1, 1), np.eye(2), samples_per_class
    )

    negative = rng.multivariate_normal(
        (-1, -1), 3 * np.eye(2), samples_per_class
    )
```

```
    features = np.vstack((positive, negative))

    labels = np.concatenate((
        np.ones(samples_per_class, dtype=int),
        -np.ones(samples_per_class, dtype=int)
    ))

    return features, labels


# Corrupt training labels
def corrupt_training_labels(clean_labels, rng):

    noisy_labels = clean_labels.copy()

    random_values = rng.random(len(clean_labels))

    positive_mask = clean_labels == 1
    negative_mask = clean_labels == -1

    noisy_labels[
        positive_mask & (random_values < 0.35)
    ] = -1

    noisy_labels[
        negative_mask & (random_values < 0.20)
    ] = 1

    return noisy_labels


# Train and evaluate SVM
def train_and_evaluate(
    train_features,
    train_labels,
    test_features,
    test_labels
):

    model = SVC(
        kernel="rbf",
        C=1.0,
        gamma=0.01
    )

    model.fit(train_features, train_labels)

    predictions = model.predict(test_features)

    return accuracy_score(test_labels, predictions)


# Part A: Baseline experiment
def run_baseline(repeats=20, seed=0):

    rng = np.random.default_rng(seed)

    # Generate datasets only once
    train_features, clean_train_labels = generate_clean_split(rng)

    test_features, test_labels = generate_clean_split(rng)

    accuracies = []

    for trial in range(1, repeats + 1):

        # Corrupt training labels
        noisy_labels = corrupt_training_labels(
            clean_train_labels, rng
        )

        # Train SVM using noisy labels
        accuracy = train_and_evaluate(
            train_features,
            noisy_labels,
            test_features,
```

```
                test_labels
            )

            accuracies.append(accuracy)

            print(f"Trial {trial}: Accuracy = {accuracy:.4f}")

        mean_accuracy = np.mean(accuracies)

        print("\nPart A Results")
        print(f"Mean Baseline Accuracy: {mean_accuracy:.4f}")

        return (
            train_features,
            clean_train_labels,
            test_features,
            test_labels,
            accuracies
        )


    if __name__ == "__main__":
        run_baseline()
```

```
Trial 1: Accuracy = 0.8345
Trial 2: Accuracy = 0.8330
Trial 3: Accuracy = 0.8440
Trial 4: Accuracy = 0.8445
Trial 5: Accuracy = 0.8395
Trial 6: Accuracy = 0.8410
Trial 7: Accuracy = 0.8405
Trial 8: Accuracy = 0.7930
Trial 9: Accuracy = 0.8240
Trial 10: Accuracy = 0.8425
Trial 11: Accuracy = 0.8220
Trial 12: Accuracy = 0.8190
Trial 13: Accuracy = 0.8470
Trial 14: Accuracy = 0.8415
Trial 15: Accuracy = 0.8160
Trial 16: Accuracy = 0.8285
Trial 17: Accuracy = 0.8235
Trial 18: Accuracy = 0.8430
Trial 19: Accuracy = 0.7840
Trial 20: Accuracy = 0.7980

Part A Results
Mean Baseline Accuracy: 0.8280
```

(b)

```
import numpy as np
import csv
from pathlib import Path
from sklearn.metrics import pairwise_distances


# K-NN label repair
def repair_labels_knn(train_features, noisy_labels, neighbors=5):

    # Calculate Euclidean distances
    distances = pairwise_distances(
        train_features,
        train_features,
        metric="euclidean"
    )

    # Exclude each point itself
    np.fill_diagonal(distances, np.inf)

    # Find five nearest neighbors
    neighbor_indices = np.argsort(
        distances,
        axis=1,
        kind="stable"
    )[:, :neighbors]
```

```
    # Get original noisy labels of neighbors
    neighbor_labels = noisy_labels[neighbor_indices]

    # Majority voting
    votes = np.sum(neighbor_labels, axis=1)

    repaired_labels = np.where(votes > 0, 1, -1)

    return repaired_labels


# Part B: Compare baseline and K-NN repair
def run_comparison(repeats=20, seed=0, neighbors=5):

    rng = np.random.default_rng(seed)

    # Generate datasets once
    train_features, clean_train_labels = generate_clean_split(rng)

    test_features, test_labels = generate_clean_split(rng)

    results = []

    for trial in range(1, repeats + 1):

        # Generate noisy labels
        noisy_labels = corrupt_training_labels(
            clean_train_labels, rng
        )

        # Baseline SVM
        baseline_accuracy = train_and_evaluate(
            train_features,
            noisy_labels,
            test_features,
            test_labels
        )

        # Repair noisy labels using K-NN
        repaired_labels = repair_labels_knn(
            train_features,
            noisy_labels,
            neighbors
        )

        # Train fresh SVM using repaired labels
        knn_accuracy = train_and_evaluate(
            train_features,
            repaired_labels,
            test_features,
            test_labels
        )

        results.append({
            "trial": trial,
            "baseline_accuracy": baseline_accuracy,
            "knn_accuracy": knn_accuracy
        })

        print(
            f"Trial {trial}: "
            f"Baseline = {baseline_accuracy:.4f}, "
            f"K-NN = {knn_accuracy:.4f}"
        )

    # Save results to CSV
    output_path = Path("results/noise_comparison.csv")
    output_path.parent.mkdir(parents=True, exist_ok=True)

    with output_path.open("w", newline="") as file:

        writer = csv.DictWriter(
            file,
            fieldnames=[
                "trial",
                "baseline_accuracy",
                "knn_accuracy"
            ]
```

```
        )

        writer.writeheader()
        writer.writerows(results)

    # Calculate mean accuracies
    baseline_mean = np.mean([
        row["baseline_accuracy"] for row in results
    ])

    knn_mean = np.mean([
        row["knn_accuracy"] for row in results
    ])

    difference = knn_mean - baseline_mean

    print("\nPart B Results")
    print(f"Mean Baseline Accuracy: {baseline_mean:.4f}")
    print(f"Mean K-NN Accuracy: {knn_mean:.4f}")
    print(f"Difference: {difference:+.4f}")

    print(f"CSV saved at: {output_path}")


if __name__ == "__main__":
    run_comparison()
```

```
Trial 1: Baseline = 0.8345, K-NN = 0.8615
Trial 2: Baseline = 0.8330, K-NN = 0.8720
Trial 3: Baseline = 0.8440, K-NN = 0.8675
Trial 4: Baseline = 0.8445, K-NN = 0.8700
```